SECURE DOCUMENT VAULT

Project Report

Data Integrity and Authentication — Final Project

Course: Data Integrity and Authentication
Project: Secure Document Vault System
Date: May 2026

Table of Contents

1. Project Overview

The Secure Document Vault is a web-based platform that enables users to securely upload, store, manage, encrypt, sign, and verify digital documents. The system integrates multiple security mechanisms to protect sensitive documents against unauthorized access, tampering, and interception.

The system was built using Python (Flask) as the backend framework, SQLite as the database, and Jinja2-rendered HTML templates for the frontend. All major security requirements have been implemented, including JWT authentication, OAuth login, Two-Factor Authentication, AES-256 document encryption, RSA digital signatures, and HTTPS support.

1.1 System Architecture

Component	Technology / Details
Backend Framework	Python — Flask with Blueprints
Database	SQLite via Flask-SQLAlchemy
Frontend	Jinja2 HTML Templates
Authentication	Flask-Login + Flask-JWT-Extended
Password Hashing	Flask-Bcrypt (bcrypt algorithm)
OAuth	GitHub OAuth 2.0 + Google OAuth 2.0
2FA	pyotp (TOTP) + qrcode
Encryption	AES-256-CBC via cryptography library
Digital Signatures	RSA-2048 with PSS padding via cryptography
HTTPS	Self-signed certificate via ssl_context adhoc (pyOpenSSL)

2. Authentication and User Management

The authentication module is implemented in app/routes/auth.py and handles all user identity management functions.

2.1 User Registration

Users register by providing a username, email address, and password. The registration endpoint validates all fields before creating the account:

- All fields are required — username, email, and password.
- Duplicate email and username checks are enforced at the database level.
- Password is validated against the password policy before the account is created.
- Passwords are hashed using bcrypt — plain text passwords are never saved.
- The first registered user is automatically assigned the Admin role.

2.2 Password Policy Enforcement

The validate_password() function enforces the following rules:

- Minimum length of 8 characters
- At least one uppercase letter (A–Z)
- At least one lowercase letter (a–z)
- At least one numeric digit (0–9)
- At least one special character: !@#$%^&*(),.?":{}|<>

2.3 Password Hashing (bcrypt)

Passwords are hashed using bcrypt via Flask-Bcrypt. bcrypt is computationally expensive by design, making brute-force attacks infeasible. The hash is stored in the password_hash column — the original password is never stored.

2.4 JWT Authentication

The system uses JSON Web Tokens via Flask-JWT-Extended to secure API endpoints. A JWT token is obtained via POST /auth/token and passed in the Authorization header for all protected API calls.

- GET /api/me — returns the authenticated user's profile
- GET /api/documents — returns the user's document list

2.5 OAuth Login (GitHub and Google)

Users can authenticate using their existing GitHub or Google accounts. The OAuth 2.0 authorization code flow is implemented manually:

1. User clicks Login with GitHub or Login with Google.
2. System redirects to the provider's authorization page.
3. Provider redirects back with an authorization code.
4. System exchanges the code for an access token and retrieves user profile.
5. Existing users are logged in; new users are registered automatically.

2.6 Two-Factor Authentication (2FA)

TOTP-based 2FA is implemented using pyotp, compatible with Google Authenticator and any TOTP app.

Setup: the user scans a QR code generated by qrcode, confirms with a 6-digit code, and 2FA is activated. On subsequent logins, the user enters the current TOTP code after the password step.

2.7 Logout

The logout endpoint calls logout_user() via Flask-Login, clears the session cookie, and records a logout event in the audit log.

3. Role-Based Access Control (RBAC)

Role	Permissions
Admin	Manage all users (activate, deactivate, delete), change roles, view all documents, view audit logs
Manager	Upload own documents, verify integrity and signatures of any document in the system
User	Upload, download, delete, and view metadata for their own documents only

Roles are stored in the roles table and linked via a many-to-many relationship (user_roles). Route-level protection uses a custom admin_required decorator and inline role checks in the document routes.

4. Secure Document Management

4.1 Upload Flow

6. File type validated against allowed extensions: pdf, png, jpg, jpeg, txt, docx, xlsx.
7. File size checked — maximum 16 MB.
8. SHA-256 hash computed from original file bytes.
9. Hash signed with user's RSA private key.
10. File encrypted with AES-256-CBC and random IV.
11. Encrypted file saved to disk with UUID-based filename (.enc).
12. Metadata (filename, hash, signature, IV, type, size) saved to database.

4.2 Operations

Operation	Description
Upload	Validates, hashes, signs, encrypts, and stores
Download	Decrypts and serves the original file
Delete	Removes encrypted file from disk and record from DB
Metadata	Shows hash, signature, upload date, verification status
Verify	Decrypts, rehashes, checks signature against stored values

5. Document Encryption (AES-256)

All uploaded documents are encrypted using AES-256-CBC before storage. A unique random 16-byte IV is generated for each upload. The AES key is derived from the application's ENCRYPTION_KEY using SHA-256. Encrypted files have the .enc extension and cannot be read without the key and IV.

- Algorithm: AES-256-CBC
- Key derivation: SHA-256 of the config ENCRYPTION_KEY
- IV: os.urandom(16) — unique per file, stored in database

- Padding: PKCS7-style to align to 16-byte block size
- Library: Python cryptography (industry-standard)

6. Digital Signatures and Integrity Verification

Each user has a unique RSA-2048 key pair stored in instance/keys/. Keys are generated automatically on first use.

6.1 Signing (on upload)

13. SHA-256 hash computed from original file bytes.
14. Hash signed with user's RSA private key using PSS padding + SHA-256.
15. Base64-encoded signature stored in the digital_signature column.

6.2 Verification

16. Encrypted file decrypted from disk.
17. SHA-256 hash recomputed and compared to stored hash.
18. Stored signature verified using user's RSA public key.
19. Result (hash match + signature validity) displayed and saved in is_verified field.

Any tampering after upload causes the recomputed hash to differ from the stored hash, and signature verification fails.

7. HTTPS and Secure Communication

The application runs over HTTPS to ensure all communication between the browser and the server is encrypted, preventing eavesdropping and man-in-the-middle attacks.

7.1 Configuration

HTTPS is enabled in run.py using Flask's built-in ssl_context support with pyOpenSSL:

```
app.run(host="127.0.0.1", debug=True, port=5000, ssl_context="adhoc")
```

The adhoc mode generates a temporary self-signed certificate automatically via pyOpenSSL on every startup. The application is accessible at https://127.0.0.1:5000. In a production deployment, this would be replaced with a certificate from a trusted Certificate Authority such as Let's Encrypt.

7.2 Protection Provided by HTTPS

- Confidentiality: all data (credentials, document content, session tokens) is encrypted in transit.
- Integrity: MACs detect any modification of data in transit.
- Authentication: the server certificate confirms the client is communicating with the genuine server.

8. MITM Simulation — Wireshark Analysis

This section demonstrates the critical importance of HTTPS through a practical traffic analysis using Wireshark. Two captures are compared: one over plain HTTP exposing credentials, and one over HTTPS showing fully encrypted traffic.

8.1 Test Environment

Wireshark was run on the Loopback interface (Adapter for loopback traffic capture) with the display filter:

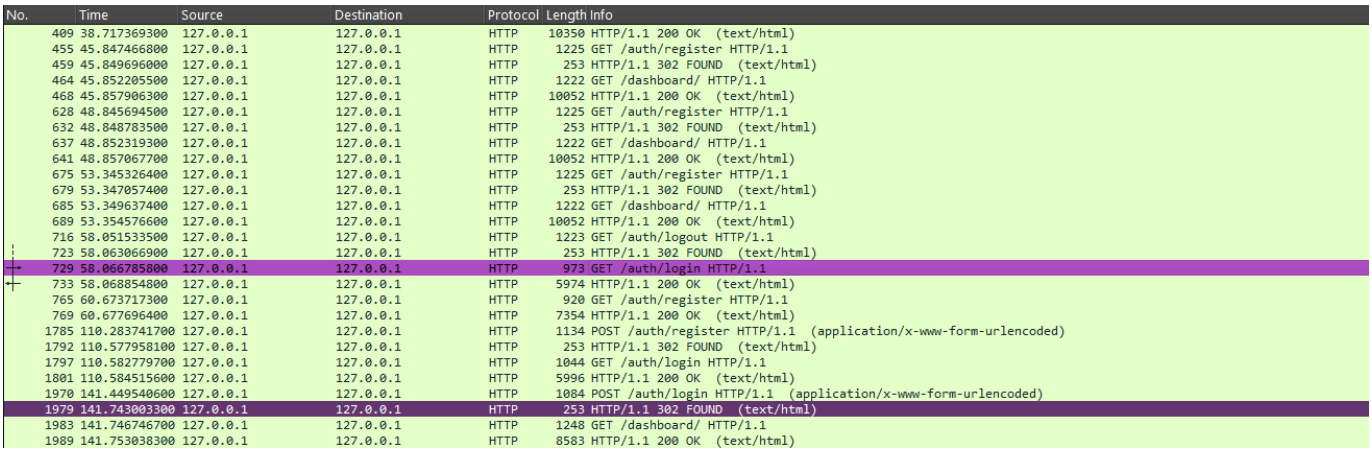
```
tcp.port == 5000
```

The same login credentials were submitted in both tests to allow direct comparison.

8.2 HTTP Capture — Credentials Exposed

In the first test, the application ran over plain HTTP. All traffic was visible in Wireshark as unencrypted HTTP packets.

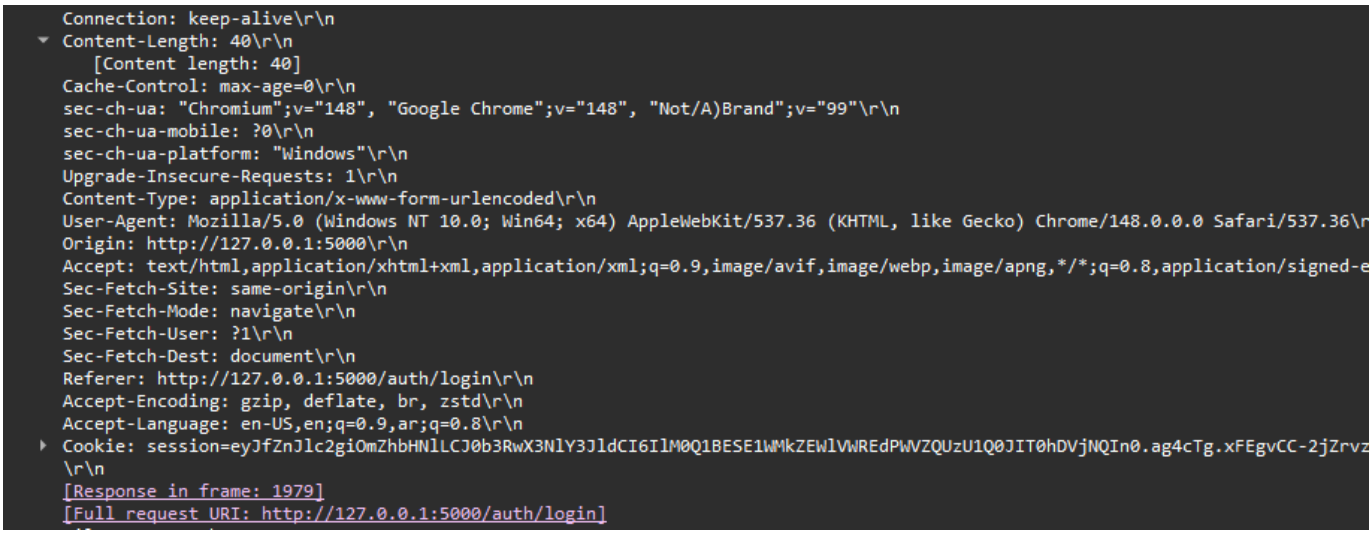
Figure 1 — HTTP traffic overview showing all endpoints in plain text:



No.	Time	Source	Destination	Protocol	Length	Info
409	38.717369300	127.0.0.1	127.0.0.1	HTTP	10350	HTTP/1.1 200 OK (text/html)
455	45.847466800	127.0.0.1	127.0.0.1	HTTP	1225	GET /auth/register HTTP/1.1
459	45.849696000	127.0.0.1	127.0.0.1	HTTP	253	HTTP/1.1 302 FOUND (text/html)
464	45.852205500	127.0.0.1	127.0.0.1	HTTP	1222	GET /dashboard/ HTTP/1.1
468	45.857906300	127.0.0.1	127.0.0.1	HTTP	10052	HTTP/1.1 200 OK (text/html)
628	48.845694500	127.0.0.1	127.0.0.1	HTTP	1225	GET /auth/register HTTP/1.1
632	48.848783500	127.0.0.1	127.0.0.1	HTTP	253	HTTP/1.1 302 FOUND (text/html)
637	48.852319300	127.0.0.1	127.0.0.1	HTTP	1222	GET /dashboard/ HTTP/1.1
641	48.857067700	127.0.0.1	127.0.0.1	HTTP	10052	HTTP/1.1 200 OK (text/html)
675	53.345326400	127.0.0.1	127.0.0.1	HTTP	1225	GET /auth/register HTTP/1.1
679	53.347057400	127.0.0.1	127.0.0.1	HTTP	253	HTTP/1.1 302 FOUND (text/html)
685	53.349637400	127.0.0.1	127.0.0.1	HTTP	1222	GET /dashboard/ HTTP/1.1
689	53.354576600	127.0.0.1	127.0.0.1	HTTP	10052	HTTP/1.1 200 OK (text/html)
716	58.051533500	127.0.0.1	127.0.0.1	HTTP	1223	GET /auth/logout HTTP/1.1
723	58.063066900	127.0.0.1	127.0.0.1	HTTP	253	HTTP/1.1 302 FOUND (text/html)
729	58.066785800	127.0.0.1	127.0.0.1	HTTP	973	GET /auth/login HTTP/1.1
733	58.068854800	127.0.0.1	127.0.0.1	HTTP	5974	HTTP/1.1 200 OK (text/html)
765	60.673717300	127.0.0.1	127.0.0.1	HTTP	920	GET /auth/register HTTP/1.1
769	60.677696400	127.0.0.1	127.0.0.1	HTTP	7354	HTTP/1.1 200 OK (text/html)
1785	110.283741700	127.0.0.1	127.0.0.1	HTTP	1134	POST /auth/register HTTP/1.1 (application/x-www-form-urlencoded)
1792	110.577958100	127.0.0.1	127.0.0.1	HTTP	253	HTTP/1.1 302 FOUND (text/html)
1797	110.582779700	127.0.0.1	127.0.0.1	HTTP	1044	GET /auth/login HTTP/1.1
1801	110.584515600	127.0.0.1	127.0.0.1	HTTP	5996	HTTP/1.1 200 OK (text/html)
1970	141.449540600	127.0.0.1	127.0.0.1	HTTP	1084	POST /auth/login HTTP/1.1 (application/x-www-form-urlencoded)
1979	141.743003300	127.0.0.1	127.0.0.1	HTTP	253	HTTP/1.1 302 FOUND (text/html)
1983	141.746746700	127.0.0.1	127.0.0.1	HTTP	1248	GET /dashboard/ HTTP/1.1
1989	141.753038300	127.0.0.1	127.0.0.1	HTTP	8583	HTTP/1.1 200 OK (text/html)

Figure 1: HTTP traffic — all request paths and methods visible

Figure 2 — Request headers of the POST /auth/login packet:



```
Connection: keep-alive\r\n
Content-Length: 40\r\n
   [Content length: 40]
Cache-Control: max-age=0\r\n
sec-ch-ua: "Chromium";v="148", "Google Chrome";v="148", "Not/A)Brand";v="99"\r\n
sec-ch-ua-mobile: ?0\r\n
sec-ch-ua-platform: "Windows"\r\n
Upgrade-Insecure-Requests: 1\r\n
Content-Type: application/x-www-form-urlencoded\r\n
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36\r
Origin: http://127.0.0.1:5000\r\n
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-e
Sec-Fetch-Site: same-origin\r\n
Sec-Fetch-Mode: navigate\r\n
Sec-Fetch-User: ?1\r\n
Sec-Fetch-Dest: document\r\n
Referer: http://127.0.0.1:5000/auth/login\r\n
Accept-Encoding: gzip, deflate, br, zstd\r\n
Accept-Language: en-US,en;q=0.9,ar;q=0.8\r\n
Cookie: session=eyJfZnJlc2giOmZhbHNlLCJ0b3RwX3NlY3JldCI6IlM0Q1BESE1WMkZEWlVWREdPWVZQUzU1Q0JIT0hDVjNQIn0.ag4cTg.xFEgvCC-2jZrvz
\r\n
[Response in frame: 1979]
[Full request URI: http://127.0.0.1:5000/auth/login]
```

Figure 2: HTTP request headers — fully readable without any decryption

Figure 3 — Full packet inspection showing the login POST with form data:

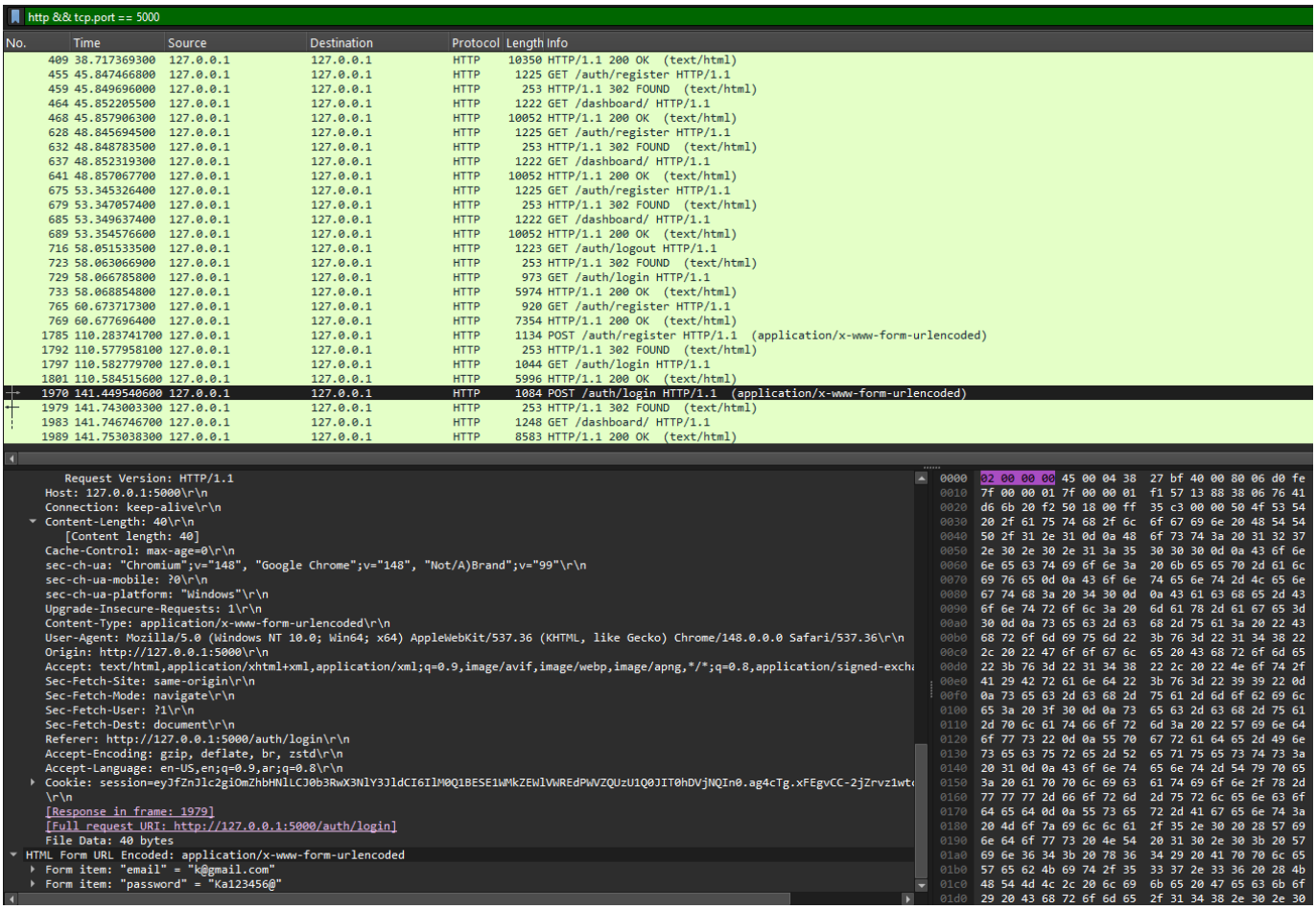


Figure 3: Full Wireshark packet — POST /auth/login with HTML Form data visible

Figure 4 — HTML Form URL Encoded data extracted from the packet:

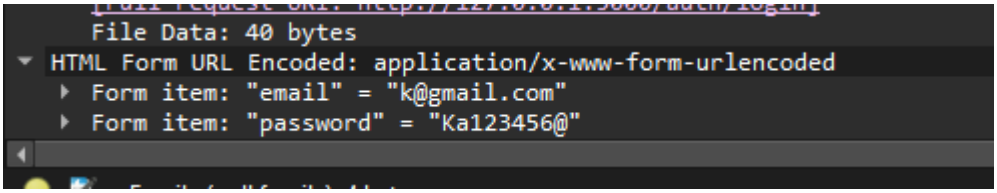


Figure 4: Form data decoded — email and password exposed in plain text

Critical Finding: Over HTTP, Wireshark clearly shows Form item: "email" = "k@gmail.com" and Form item: "password" = "Ka123456@" — credentials are fully readable by any network observer with no special tools required.

Figure 5 — The login page as submitted by the user:

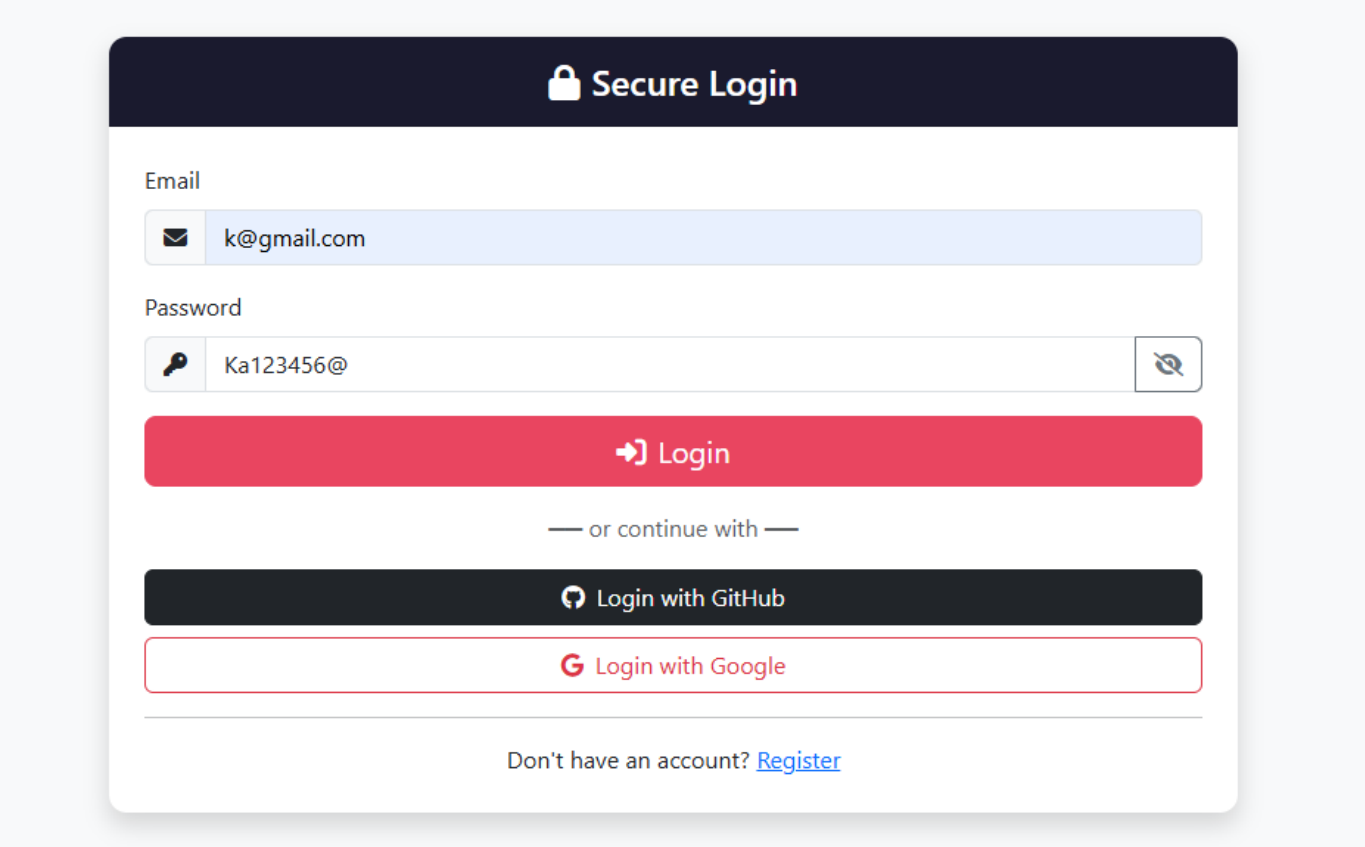


Figure 5: Login page — credentials entered by the user, transmitted in plain text over HTTP

8.3 HTTPS Capture — Traffic Encrypted

In the second test, the application was switched to HTTPS mode using ssl_context="adhoc" in run.py. The same login action was performed and Wireshark was used to capture traffic on the same interface with the same filter.

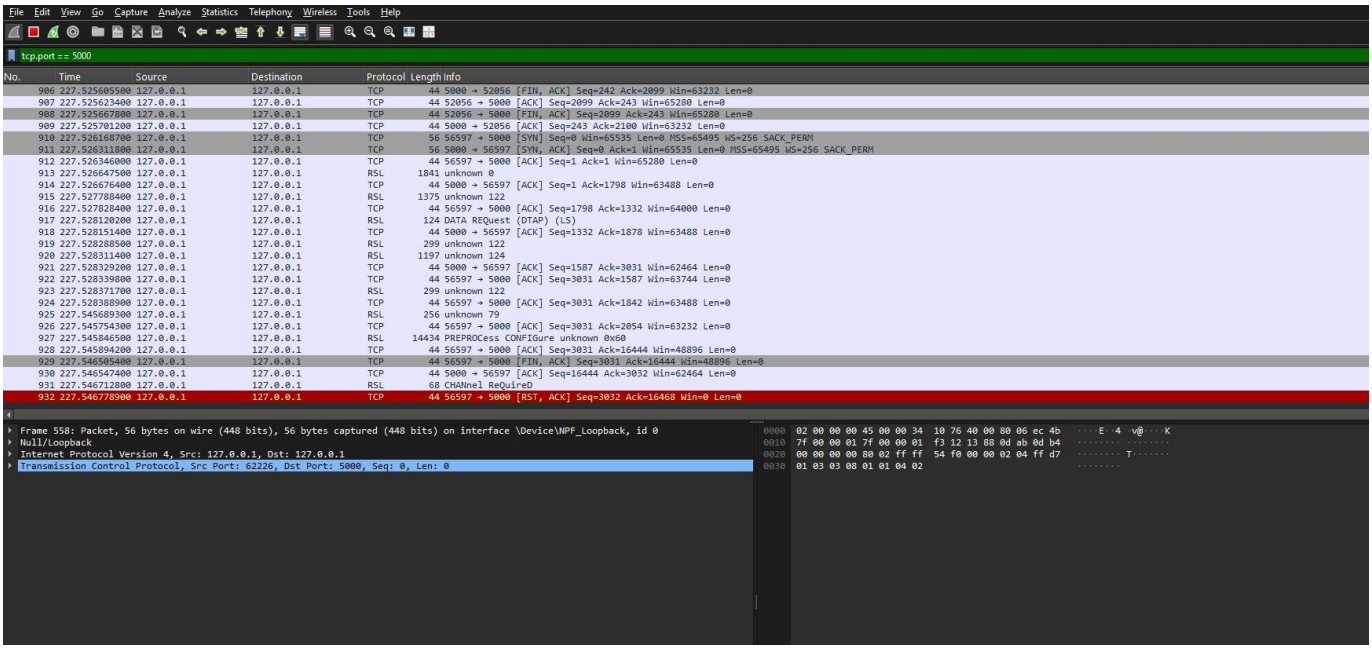


Figure 6: Wireshark capture over HTTPS — protocol shows TCP/RSL, no readable data

Result: After enabling HTTPS, Wireshark shows only TCP and RSL (TLS Record Layer) packets. The protocol column shows no HTTP entries. All payload data appears as encrypted, unreadable bytes — credentials, session tokens, and document content are completely protected from interception.

8.4 HTTP vs HTTPS Comparison

	HTTP (Unencrypted)	HTTPS (Encrypted)
Protocol in Wireshark	HTTP — fully readable	TCP / RSL (TLS) — encrypted
Login credentials	email + password in plain text	Encrypted — unreadable
Session cookies	Visible — can be hijacked	Encrypted and protected
Document content	Transmitted in plain text	Encrypted in transit
MITM susceptibility	Trivially easy to intercept	TLS prevents interception
Wireshark can read data?	Yes — full form data visible	No — only ciphertext visible

9. Audit Logging

The system records all security-relevant actions in the audit_logs table with: user ID, action type, target, IP address, timestamp, and details.

- login / login_2fa / failed_login
- logout
- upload (with first 16 chars of SHA-256 hash)

- download / delete
- verify (with hash_match and sig_valid results)

The Admin panel provides a dedicated Audit Logs page showing the 50 most recent events.

10. Requirements Compliance Summary

Requirement	Implementation	Status
User registration and login	Flask-Login, bcrypt	✓ Done
Password hashing	bcrypt via Flask-Bcrypt	✓ Done
Password policy enforcement	validate_password() — 5 rules	✓ Done
JWT authentication	Flask-JWT-Extended	✓ Done
OAuth — GitHub	OAuth 2.0 authorization code flow	✓ Done
OAuth — Google	OAuth 2.0 authorization code flow	✓ Done
Two-Factor Authentication	pyotp TOTP + QR code	✓ Done
Logout	Flask-Login logout_user()	✓ Done
RBAC — 3 roles	Admin / Manager / User	✓ Done
Admin panel	User management, roles, audit logs	✓ Done
Document upload	Validation + encrypt + sign + store	✓ Done
Document download	Decrypt and serve	✓ Done
Document delete	Remove file and DB record	✓ Done
Document metadata	Dedicated metadata page	✓ Done
File type validation	7 allowed types enforced	✓ Done
File size validation	16 MB maximum	✓ Done
AES-256 encryption	AES-256-CBC with random IV per file	✓ Done
SHA-256 hash	Computed on every upload	✓ Done
Digital signatures	RSA-2048 PSS — per user key pair	✓ Done
Integrity verification	Hash recompute + signature verify	✓ Done
HTTPS	ssl_context adhoc via pyOpenSSL	✓ Done
Wireshark HTTP demo	Credentials exposed in plain text	✓ Done
Wireshark HTTPS demo	Traffic encrypted — RSL/TLS only	✓ Done
Web-based UI	6 page types — complete interface	✓ Done
Audit logging	All security events logged	✓ Done

11. How to Run the Project

11.1 Prerequisites

- Python 3.9 or higher
- pip
- pyOpenSSL (for HTTPS)

11.2 Installation

20. Extract the project files.
21. Install dependencies:

```
pip install -r requirements.txt
```

22. Install pyOpenSSL for HTTPS:

```
pip install pyopenssl
```

23. Copy and edit environment file:

```
cp .env.example .env
```

24. Run the application:

```
python run.py
```

25. Open browser at: https://127.0.0.1:5000
26. Register first account — it automatically receives the Admin role.

11.3 Environment Variables

Variable	Description
SECRET_KEY	Flask session secret key
JWT_SECRET_KEY	JWT signing key
ENCRYPTION_KEY	AES encryption key (32+ bytes)
GITHUB_CLIENT_ID / SECRET	GitHub OAuth App credentials
GOOGLE_CLIENT_ID / SECRET	Google OAuth credentials

12. Conclusion

The Secure Document Vault system successfully implements all required security features. The application demonstrates practical integration of modern security concepts including multi-factor authentication, encrypted storage, digital signatures, role-based access control, and secure transport via HTTPS.

The Wireshark demonstration provides clear, evidence-based proof of the risk posed by unencrypted HTTP and the protection offered by HTTPS. The captured credentials in plain text

(Figure 4) versus the fully encrypted TLS traffic (Figure 6) present a compelling real-world comparison that illustrates why HTTPS is mandatory for any system handling sensitive information.